

Mizan AI

System documentation · generated locally

SYSTEM

Overview

System Architecture

LangGraph Workflows

RECOMMENDATION AGENT

Agent Design

Tools Available

Weight Assignment

The 8 Criteria

Scoring Mathematics

Cost Model

DECISIONING

Eligibility Filter

Cohort Assignment

Verification & Routing

Clarify Safety

Deterministic Fallback

INTERNAL DOCUMENTATION

Mizan AI — System & Recommendation Agent

A LangGraph-orchestrated intake → assessment → recommendation pipeline for a UAE health-insurance broker. This page documents the system architecture, the two LangGraph state machines, and — in full mathematical detail — the ReAct-style recommendation agent: its tools, its weight assignment, and the arithmetic that turns weights into a ranked shortlist.

Runtime: `Next.js 16` · `LangGraph 1.4` · `OpenRouter` Data: `SQLite` via `Drizzle ORM` Source: `lib/recommendation/*`, `lib/ai/*`

01 Overview

How the pieces fit together, before the deep dive.



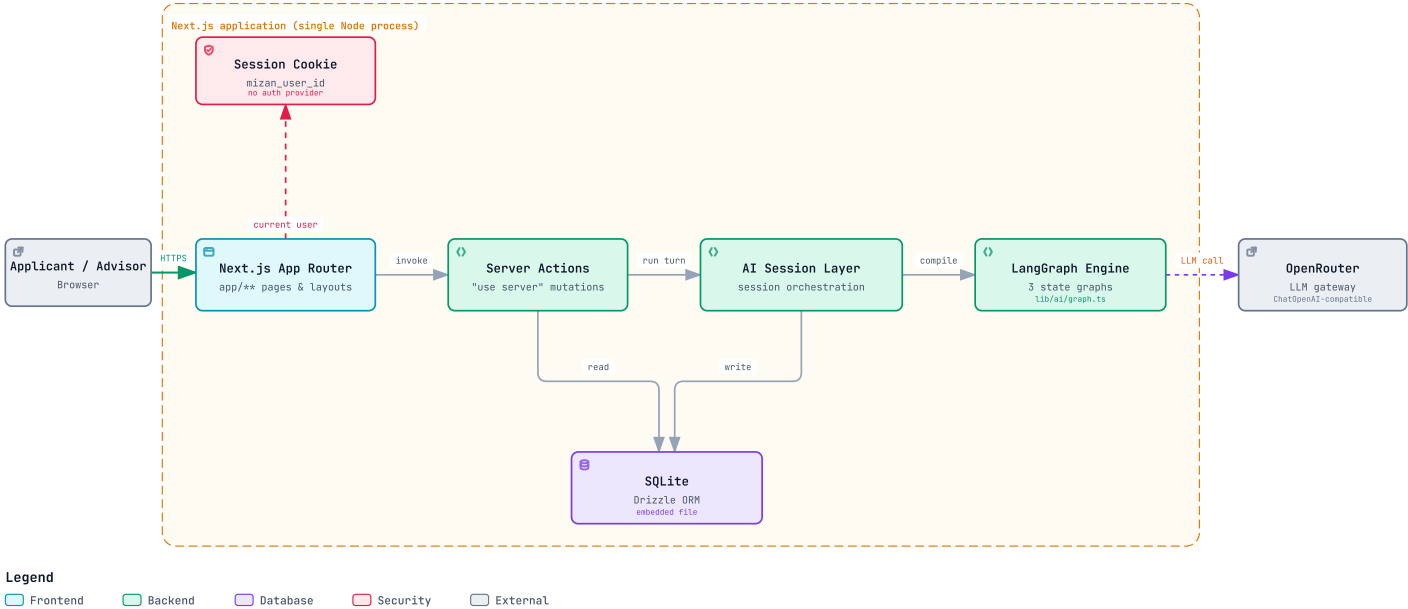
Every application passes through three LangGraph state machines, compiled and invoked separately but sharing one record: **intake** (turning a conversation into a structured application), **assessment** (validating, classifying into a cohort, and routing flagged records to an advisor), and **recommendation** (pricing all plans and building a ranked, weighted shortlist). Recommendation does not redeclare its own state — its nodes attach directly to `AssessmentState` and read the `record`, `catalogue`, and `cohort` that `validate`/`classify` already populated for that application.

The recommendation agent is the most involved piece: rather than a single scored formula applied blindly, an LLM-driven agent *chooses which of 8 comparison criteria matter for this specific applicant and how much*, within hard numeric bounds, and a deterministic scoring engine — never the model — turns those weights into a ranked list. Every number the agent states in its final reasoning must trace back to a tool observation it actually received; anything it can't cite is caught at verification and routed to a human.

02 System Architecture

Full-system component diagram — embedded live from `diagrams/architecture.html`.

Mizan AI – System Architecture



Intake

- Chat or static-form intake under `app/applications/new/**`
- `converse` → `ask` / `confirm` graph extracts and settles fields

Assessment & recommendation

- `validate` → `classify` → `narrate` → `route`, with an advisor gate
- `price` → `recommend` → `verify`, then `clarify` or `gate` before presenting plans

Model calls & persistence

- OpenRouter via a single ChatOpenAI client, with retries and a deterministic scripted fallback with no API key
- SQLite via Drizzle ORM, one embedded file – no real auth yet, the active user is a cookie

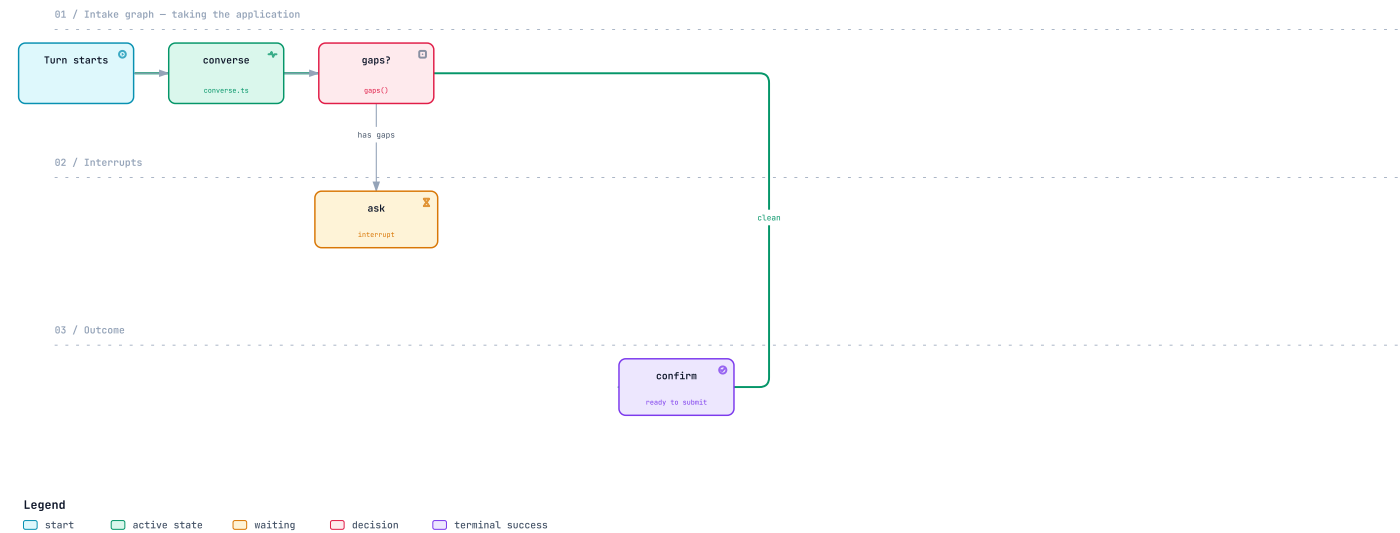
03 LangGraph Workflows

The two LangGraph state graphs at node/edge level — node shape encodes role (start, active step, decision, human interrupt, success).

Intake graph — taking the application

[open standalone ↗](#)

Mizan AI – Intake Graph (LangGraph)



Taking the application

- converse extracts fields from the applicant's reply and drafts a response
- gaps() checks what is still missing after every turn

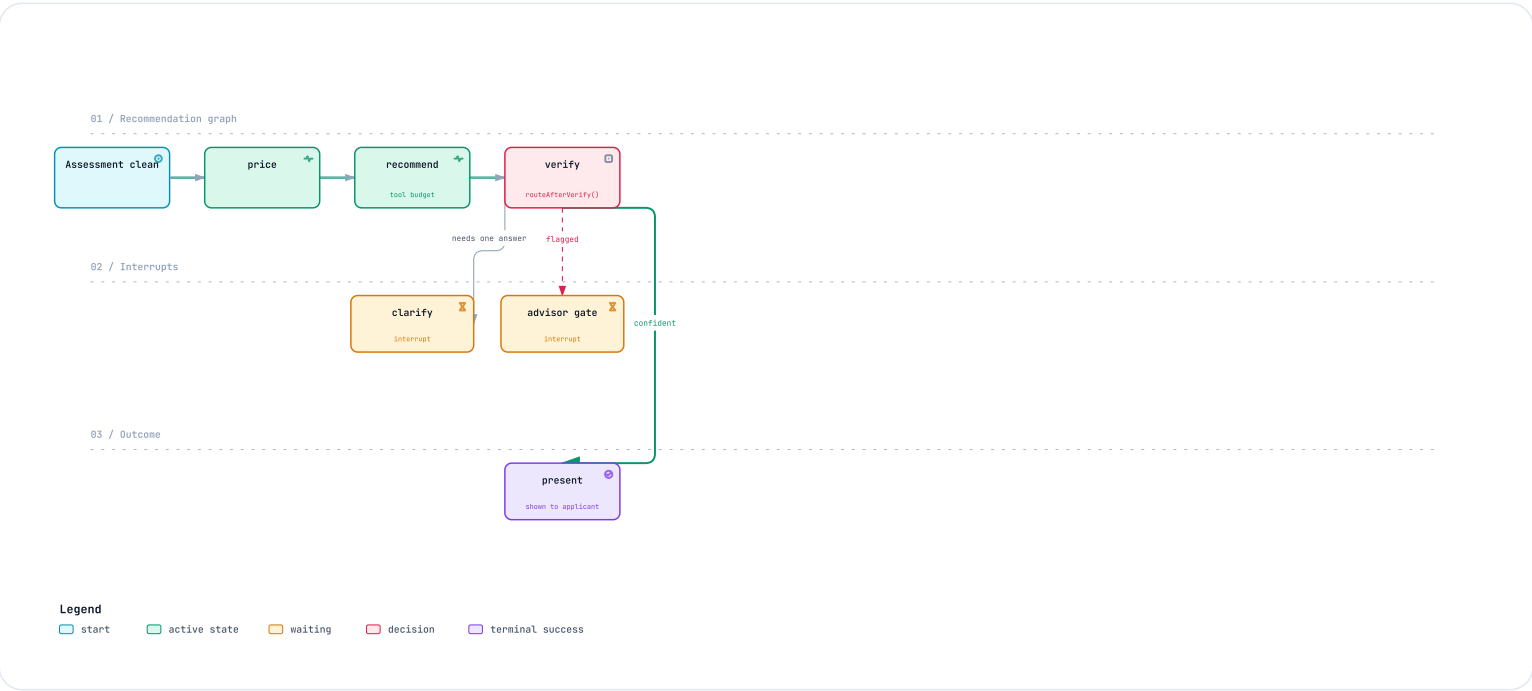
An interrupt, not an error

- ask pauses the graph and hands control back to the applicant
- Nothing is lost – the conversation log in SQLite is durable state, replayed on the next turn

Recommendation graph

[open standalone ↗](#)

Mizan AI – Recommendation Graph (LangGraph)



Pricing and shortlisting

- price quotes every plan in the catalogue for this cohort
- recommend lets an agent build a shortlist over a bounded tool budget

Two interrupts, one gate

- clarify asks the applicant one question when the shortlist is unclear
- advisor gate hands a flagged shortlist to a person – neither is a failure, just a pause

04 Recommendation Agent — Design

How `lib/ai/graph/nodes/recommend.ts` is built, and why it looks the way it does.

Not native function calling — a JSON action loop

`lib/ai/openrouter.ts` is explicit that the free OpenRouter models this app runs on advertise neither strict structured output nor reliable native tool calling. So the agent is a hand-rolled **ReAct-style JSON action loop** built on the same `structuredCall` primitive every other node in the system uses: each turn the model is asked for exactly one JSON object —

```
{ "thought": "...", "tool": "...", "args": { ... } }
```

— the runtime validates the call against the closed vocabulary tables in `lib/ai/tools/plans.ts`, executes it, and appends the observation to the transcript as plain text for the next turn. The tool *names* and *schemas* are the real contract; if `OPENROUTER_MODEL` is later pointed at a tool-capable model, the same registry binds natively via `bindTools` with no change to the tools themselves.

The budget

- **8 ordinary tool calls** per round (`MAX_TOOL_CALLS`). Each turn's system prompt reminds the model how many calls remain once 3 or fewer are left.
- If the model sends the **same invalid value for the same tool 3 times in a row** (`MAX_SAME_ERROR_REJECTIONS`), the round ends immediately and falls back to the deterministic recommender. A wrong value corrected on the very next turn is *not* punished — only the identical mistake repeated.
- If the budget is spent with no `propose_shortlist` call, there is **one forced final turn**: the tool list narrows to `propose_shortlist` alone, so the model must decide from what it already gathered rather than reach for one more lookup it has no room for.
- If that forced turn also fails, or no model is configured at all (`isAgentEnabled() === false`), control passes to the deterministic fallback — see §13.

DESIGN INVARIANT

The agent is **never handed the plan corpus and supplies no numbers of its own**. Every amount, waiting period, or figure it reasons about must come from a tool's observation. This is enforced twice: once by the system prompt, and once, unconditionally, by `verify`'s citation check (§11) after the agent has finished.

Pipeline position

Recommendation is three deterministic/agentic stages in sequence, all attached to the one shared `AssessmentState`:



`price` is pure arithmetic — it quotes every plan up front so the applicant-facing comparison and the agent's own reasoning are always working from the same frozen premiums. `recommend` is the agent loop described above. `verify` is pure arithmetic again — a hard eligibility re-check plus the citation check — and cannot be influenced by anything the model said. Only after `verify` does a conditional edge decide whether the applicant sees the shortlist directly, is asked one clarifying question, or the application goes to an advisor.

05 Tools Available

The agent's complete tool registry — `lib/ai/tools/plans.ts`. Every argument is validated against a closed vocabulary before the tool runs; a bad value returns a structured, correctable error, never a thrown exception.

TOOL	ARGUMENTS	RETURNS
read_applicant_record	none	The declared record, cohort label, and any flags already on file.
list_plan_summaries	none	All 3 catalogue plans: id, name, premium, deductible, network, copay %, annual limit, dental tier. The entry point — tiny on purpose.
get_plan_terms	<code>{ planIds: string[], benefitClasses?: BenefitClass[] }</code>	Deeper terms for named plans — only the benefit-class slices requested (maternity / chronic_preexisting / dental_optical / general), never a full dump.
check_need_against_plans	<code>{ needId }</code> or <code>{ benefitClass, horizonMonths }</code>	Per plan: <code>covered</code> , <code>usable</code> (covered and the wait clears inside the horizon), <code>waitMonths</code> , and a human-readable <code>whyNot</code> .

TOOL	ARGUMENTS	RETURNS
check_network_access	{ planId }	Every provider on the applicant's record, admitted or refused under that plan's network tier, and why.
estimate_annual_cost	{ planId, scenarioId }	A full cost breakdown (§7) for one of 5 named scenarios. The agent names a scenario id — it never supplies visit counts or amounts; the basket is built server-side and its provenance returned alongside the number.
previous_rounds	none	Prior shortlists this applicant already saw and rejected, and the stated reason (round 2+ of the same application only).
score_plans	{ criteria: [{ criterionId, weight }] }	The full multi-criteria ranking — this is the tool that runs the scoring mathematics in §8: raw values, per-criterion normalisation, contributions, weighted score, and rank for every plan.
propose_shortlist	{ picks, rejections, confidence, uncertaintyReason?, brokerReasoning, memberReasoning }	Terminal — ends the loop. Ineligible picks are stripped server-side regardless of what the model returned. brokerReasoning memberReasoning are the two durable registers verify citation-checks.

WHY DESCRIPTIONS ARE GENERATED, NOT HAND-WRITTEN

Every tool description shown to the model (describeTools()) is built from the *live* vocabulary tables for that specific applicant — the exact plan ids in the catalogue, the exact need ids on this record, the exact benefit-class enum. A hand-written prompt that once omitted this made get_plan_terms and check_need_against_plans effectively unguessable to the model, and drove every round into a rejection-triggered fallback before it was fixed.

06 Weight Assignment

The agent decides *what* matters and *how much* — the direction and arithmetic never move.

Weights are not configured, tuned offline, or fixed per cohort. On every recommendation round, the agent itself chooses a subset of the 8 criteria (§7) and assigns each a weight, by calling the `score_plans` tool. That choice is constrained, not free-form:

CONSTRAINT	VALUE	ENFORCED BY
Minimum weight per criterion	0.05	<code>MIN_WEIGHT</code>
Maximum weight per criterion	0.6	<code>MAX_WEIGHT</code>
Maximum criteria per call	5	<code>MAX_CRITERIA</code> — "an agent that weights everything has prioritised nothing"
Minimum criteria per call	1	<code>scorePlans()</code> throws on an empty list
No duplicate criterion in one call	—	<code>scorePlans()</code> throws on a repeated <code>criterionId</code>
Criterion must be relevant to this record	—	each criterion's own <code>isRelevant()</code> predicate — see §7

A call that violates any of these returns a structured tool error the agent can read and correct — the same "wrong value → error → retry, or repeat and fall back" discipline every tool in the registry follows (§4). Once accepted, the raw weights are **normalised to sum to 1** before they touch the scoring arithmetic — see §8's first equation. The unnormalised weights the agent actually sent are preserved separately in the result (`rawWeights`) purely for audit/trace purposes; they never drive the score directly.

WHAT "ASSIGNING A WEIGHT" ACTUALLY CHANGES

A weight never touches a criterion's *direction* (whether more of it is good or bad) or its raw-value formula — those are fixed in code (§7) and a prompt cannot move them. The agent's only lever is *how much each already-defined signal counts* for this specific applicant, and it can name at most 5 of the 8 available signals per round.

07 The 8 Criteria

`lib/recommendation/score.ts` — the closed vocabulary `score_plans` validates every weight against.

CRITERION	DIRECTION	RELEVANT WHEN...	RAW VALUE <code>V(PLAN)</code>
premium_cost	LOWER IS BETTER	always	the plan's <code>annualPremium</code>
total_annual_outlay	LOWER IS BETTER	always	<code>estimateAnnualCost(plan, MEDIUM_OUTPATIENT).total</code> — premium + expected out-of-pocket under a typical-use basket (\$9)
need_coverage	HIGHER IS BETTER	the record declares ≥ 1 need	count of the applicant's declared needs this plan covers, anywhere
waiting_period_fit	HIGHER IS BETTER	≥ 1 need has both a benefit class <i>and</i> a horizon	count of those needs whose waiting period clears inside the applicant's stated horizon, on this plan
network_access	HIGHER IS BETTER	the record names ≥ 1 preferred provider	count of named providers admitted under this plan's network tier
chronic_depth	HIGHER IS BETTER	the record declares ≥ 1 condition	<code>chronicCovered ? 100 / (1 + waitMonths) : 0</code> — covered-with-no-wait scores highest, not-covered scores 0
annual_limit_headroom	HIGHER IS BETTER	always	the plan's <code>annualLimit</code>
dental_optical	HIGHER IS BETTER	a declared priority's free text mentions "dental" or "optical"	<code>full → 2, basic → 1, none → 0</code>

Note the deliberate overlap between `premium_cost` and `total_annual_outlay`: the first is sticker price, the second is a modelled real-world exposure (premium *plus* deductible *plus* expected co-pay). An agent can weight either, both, or neither — they are independent signals the arithmetic never conflates.

08 Scoring Mathematics

The exact arithmetic `scorePlans()` runs — deterministic, and identical every time for the same weights and the same plan panel.

Let P be the panel of plans being compared, and let the agent have supplied weights for a set of criteria $\{1, \dots, k\}$, $k \leq 5$.

1 — Normalise the weights

WEIGHTS ARE RESCALED TO SUM TO 1 BEFORE ANYTHING ELSE HAPPENS

$$w'_i = \frac{w_i}{\sum_{j=1}^k w_j}, \quad 0.05 \leq w_i \leq 0.6, \quad k \leq 5$$

2 — Normalise each criterion's raw values across the panel

MIN-MAX NORMALISATION, PER CRITERION, OVER THE PLANS ACTUALLY BEING COMPARED

$$n_i(p) = \begin{cases} \frac{v_i(p) - \min_{q \in P} v_i(q)}{\max_{q \in P} v_i(q) - \min_{q \in P} v_i(q)} & \text{if } \max_q v_i(q) \neq \min_q v_i(q) \\ 1 & \text{if every plan ties on this criterion} \end{cases}$$

This is why a criterion measured in AED (`premium_cost`) and one measured in months or a plan count can be weighted against each other at all — comparing raw units directly would let whichever criterion happens to have the biggest numbers dominate, regardless of the weight it was actually given.

3 — Flip direction where "lower" means "better"

APPLIED ONLY TO `PREMIUM_COST` AND `TOTAL_ANNUAL_OUTLAY` — THE TWO LOWER-IS-BETTER CRITERIA

$$n_i(p) \leftarrow 1 - n_i(p) \quad \text{if criterion } i \text{ is lower_is_better}$$

After this step, $n_i(p) \in [0, 1]$ means the same thing for every criterion regardless of its native direction: 0 is the worst plan on the panel for that signal, 1 is the best.

4 — Weighted score, and the per-criterion contribution

EACH CRITERION'S CONTRIBUTION TO A PLAN'S SCORE, AND THE TOTAL

$$\text{contribution}_i(p) = w'_i \cdot n_i(p)$$
$$S(p) = \sum_{i=1}^k w'_i \cdot n_i(p)$$

Every contribution is kept alongside the total in the tool's result (`ScoredPlan.contributions`) — the agent (and the broker view) can see not just that a plan scored 0.71, but exactly how much of that came from each criterion it chose to weight.

5 — Rank

PLANS ARE SORTED STRICTLY BY WEIGHTED SCORE, DESCENDING

$$\text{rank}(p) = 1 + \#\{q \in P : S(q) > S(p)\}$$

WORKED SHAPE

Suppose the agent weights `premium_cost: 0.4`, `need_coverage: 0.35`, `network_access: 0.25` — these already sum to 1, so $w' = w$ unchanged. For each plan, its premium is min-max-normalised across the 3-plan panel and flipped (cheapest $\rightarrow 1$, priciest $\rightarrow 0$); its need-coverage count and provider-admit count are each min-max-normalised directly (already higher-is-better). The three normalised values are multiplied by 0.4 / 0.35 / 0.25 respectively and summed into one $S(p) \in [0, 1]$ per plan, and the plan with the highest $S(p)$ ranks first.

09 Cost Model

`lib/recommendation/cost.ts` & `scenarios.ts` — pure arithmetic behind `total_annual_outlay` and the `estimate_annual_cost` tool.

A cost estimate applies a usage `basket` (a named scenario) to one plan's terms: the deductible is spent first, then the co-pay percentage applies to whatever remains.

GROSS UTILISATION COST, DEDUCTIBLE, MEMBER CO-PAY, AND TOTAL EXPOSURE

$$G = V \times 350 + A \times 18,000$$
$$D = \min(\text{deductible}, G)$$
$$C = \text{round}\left((G - D) \times \frac{\text{copay}\%}{100}\right)$$
$$T = \text{round}(\text{premium} + D + C)$$

where V = outpatient visits, A = inpatient admissions in the basket, 350 AED/visit and 18,000 AED/admission are declared modelling constants (`constantsVersion: "scenarios-v1"`), not real claims data — flat, no loading, and always labelled with the scenario and constants version that produced them.

The 5 named scenarios

SCENARIO ID	BASKET	SELECTABLE WHEN...
LOW_OUTPATIENT	3 outpatient visits	always
MEDIUM_OUTPATIENT	8 outpatient visits	always — this is the basket <code>total_annual_outlay</code> and the deterministic fallback both use
HIGH_OUTPATIENT	18 visits, <code>chronic_preexisting</code>	only if the record declares ≥ 1 condition — otherwise the scenario would assert a fact the record doesn't contain
EXPECTED_INPATIENT	8 visits + 1 admission	always
CUSTOM_FROM_APPLICANT	8 visits; +1 admission if a maternity need is present; benefit classes = the union of the applicant's own dated needs' classes	only if ≥ 1 need has both a benefit class <i>and</i> a horizon — the tool returns exactly which need row(s) it was derived from, so the provenance sits in the trace next to the number it produced

The agent names a scenario id and nothing else — it never supplies a visit count or an amount. The basket is always assembled server-side.

10 Eligibility Filter

`lib/recommendation/eligibility.ts` — a coverage test, not a budget one.

A PLAN IS ELIGIBLE FOR A RECORD IFF IT COVERS EVERY BENEFIT CLASS THE RECORD DECLARED A NEED FOR

$$\text{eligible}(p, r) = \bigwedge_{n \in r.\text{needs}} \left(n.\text{benefitClass} = \emptyset \vee \text{covers}(p, n.\text{benefitClass}) \right)$$

A plan above the applicant's stated budget band is **still eligible** — that's a cost tradeoff for the scoring to weigh (via `premium_cost` / `total_annual_outlay`), not a hard exclusion. The only rule that hard-excludes on budget alone is `budget_below_cheapest_plan` (assessment-time, not here): if literally nothing on the panel is affordable, the whole application routes to a human before recommendation ever runs.

This same predicate is applied **twice**, independently: once inside `propose_shortlist` mid-loop (an ineligible pick is stripped from whatever the agent returned), and once again, unconditionally, inside `verify` (§11) — catching both agent drift and the deterministic fallback path.

11 Cohort Assignment

`lib/assessment/cohort.ts` — a routing label, not a verdict. Deterministic, tested in order, broker-only vocabulary that never reaches the applicant.

The cohort doesn't feed `score_plans` directly — the agent is free to weight any relevant criterion regardless of cohort. It matters in two places: it's the axis a human advisor reasons from in the review queue, and it's the **tie-break rule** the deterministic fallback (§13) uses when there's no agent judgement to fall back on.

COHORT	TEST (IN ORDER)	WHAT DECIDES THIS RECORD
maternity_planning	a maternity need with horizon ≤ 12 months — tested <i>first</i> , overrides everything below	waiting-period arithmetic, not premium band
chronic_complex_senior	≥1 declared condition, age ≥ 60	depth of cover and annual limit outrank premium
chronic_unstable	≥1 declared condition, any marked <code>unstable</code>	utilisation is unpredictable; the chronic waiting period lands differently
chronic_managed_mature	≥1 declared condition, stable, age ≥ 45	the chronic waiting period is the axis
chronic_managed_adult	≥1 declared condition, stable, age < 45	chronic cover matters; expected utilisation still low
standard_young_healthy	no conditions, age < 35	price-led placement
standard_mid_career	no conditions, 35 ≤ age < 60	network access and outpatient terms, not premium

COHORT	TEST (IN ORDER)	WHAT DECIDES THIS RECORD
standard_senior_healthy	no conditions, age ≥ 60	healthy today, but the age band carries utilisation risk the record doesn't show yet

12 Verification & Routing

`lib/ai/graph/nodes/verify.ts` + `routeAfterVerify()` — deterministic checks over whatever the agent (or the fallback) produced, before anything is persisted or shown to anyone.

Two checks, both load-bearing

- **Hard filter** — the shortlist is re-run through `isEligible` (\$10); any plan that fails is stripped regardless of what proposed it.
- **Citation check** — every number in `brokerReasoning` and `memberReasoning` must appear among the numbers actually observed in a *successful* tool call this round. A figure with no matching observation is treated as a hallucination, not a rounding difference. *(Skipped when the result came from the deterministic fallback — that prose is templated straight from the same arithmetic that ranked the plans, not generated by a model with room to invent.)*

VERIFYFAILED IS TRUE WHEN ANY OF THESE HOLD

$$\text{verifyFailed} = (|\text{stripped}| > 0) \vee (|\text{uncited}| > 0) \vee (|\text{validShortlist}| = 0)$$

A failed verification unconditionally forces `recoConfidence` to `"low"`, regardless of what the agent originally reported.

The routing decision

CONDITION	ROUTE
fell back to the deterministic recommender, or verification failed	ADVISOR GATE
confidence is <code>Low</code> , and the applicant hasn't been asked a clarifying question yet this application	CLARIFY — one targeted question

CONDITION	ROUTE
confidence is <code>low</code> , and one clarifying question was already asked	ADVISOR GATE
otherwise (clean, confident)	PRESENT — shown to the applicant directly

The advisor gate is **informational**, not a hold: the applicant is never blocked from seeing a clean shortlist while an advisor quietly audits it in parallel. Withholding a plan they might have accepted anyway is judged worse than a quiet audit trail — the hard filters have already run by this point regardless.

13 Clarify Safety

`lib/ai/graph/nodes/clarify.ts` — when the agent is genuinely, honestly unsure, ask the applicant one targeted question instead of going straight to a human. Every safeguard below is deterministic; no second model call judges the first one.

A clarifying question's `target` is always one of the same 8 `CriterionId`s from §7 — nothing new is invented; a clarification can only ever point at something the scoring engine already knows how to act on.

`validateClarification()` runs every check below *before* the applicant ever sees the question:

- `target` must be one of the 8 known criterion ids.
- `target` must be relevant to this record (the same `isRelevant()` gate from §7).
- if `score_plans` was called this round, `target` must be among the criteria it actually weighted — not just anything theoretically relevant.
- the question must be non-empty, ≤ 200 characters, and contain at most one `?` — a questionnaire in disguise fails here.
- the question may not name any specific plan id or plan name — steering, not asking.
- every number appearing in the question must already have been observed this round (same citation discipline as §12, applied to the question itself).
- the question may not touch a medical/PII blocklist (*diagnosis, medication, prescription, passport, bank account, social security, condition name*).

FAIL-CLOSED

Any single failure above — or the question-generation call itself failing — degrades into the exact same advisor-gate interrupt a live model failure already takes. There is no separate "clarification failed" state; a rejected clarification is indistinguishable,

downstream, from an ordinary advisor gate. At most one clarifying question is ever asked per application.

14 Deterministic Fallback

`lib/recommendation/fallback.ts` — no API key, unparseable model output, 3 repeated identical validation failures, or a budget-exhausted agent all land here. The app runs end to end with no model configured at all.



1 — Pool

Start from plans `price` already marked `eligible`; if that set is somehow empty, fall back to `shortlistablePlans` directly.

2 — Event-need filter

Plans that don't clear every declared `dated (event)` need in time are dropped — a one-off event whose waiting period doesn't clear in time isn't served by that plan in any way that helps. A `continuous` need (ongoing chronic management) behind a wait is still real cover and stays in contention. The filter is skipped if applying it would empty the pool entirely.

3 — Pick by cohort (the tie-break)

This reads the cohort's own stated rationale (§11) as arithmetic instead of prose:

COHORT	WINNER
chronic_complex_senior, chronic_unstable	plan with the highest <code>annualLimit</code>
standard_mid_career, standard_senior_healthy	the <code>standard</code> -network plan if one exists on the candidate list, else cheapest premium
everything else	cheapest premium — <code>standard_young_healthy</code> is explicitly price-led; <code>maternity_planning</code> is already event-filtered to what clears in time; the managed-chronic cohorts treat "covered, with an acceptable wait" as enough, and let price decide the rest

4 — Prose, not judgement

Both reasoning registers are templated directly from the numbers this arithmetic just produced — never run through a model, so there is nothing for a model failure to break here. Confidence is unconditionally `"Low"`, and `uncertaintyReason` always reads *"No model judgement was applied — this is a rule-based placement."* — which is exactly what routes it to the advisor gate in §12.

Generated locally from the current state of `lib/recommendation/*`, `lib/assessment/*`, and `lib/ai/*`. Not a live/auto-refreshing view — re-run to reflect code changes.